CLOUD

Kubernetes Architecture

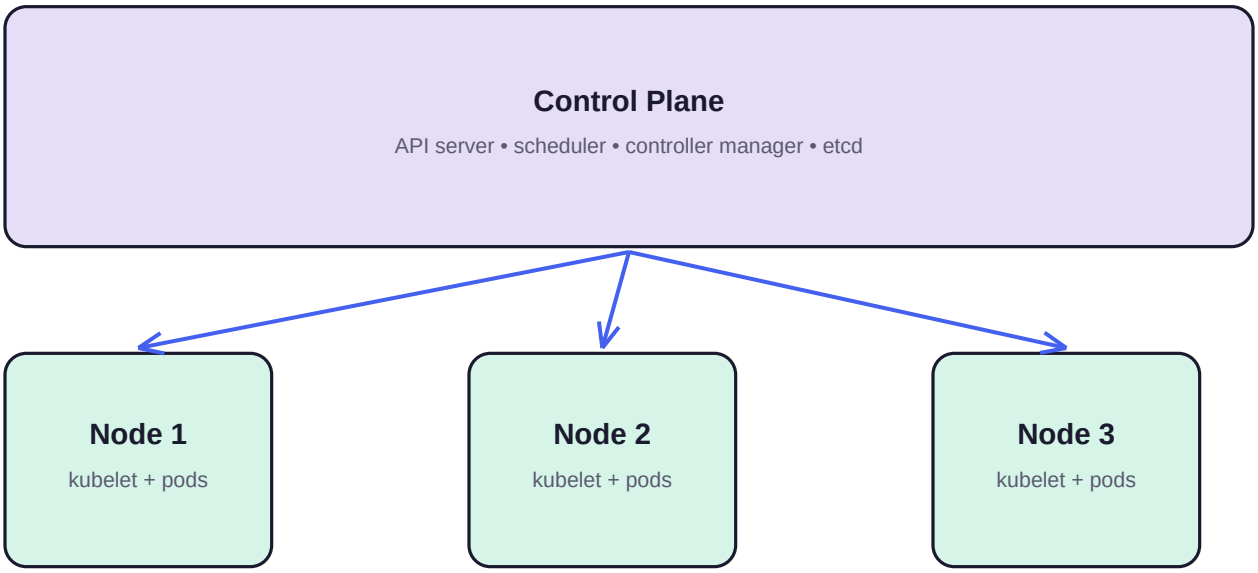
Pods, Services, Deployments —
how the orchestrator runs your containers.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Big Picture: Control Plane + Nodes

You declare desired state; Kubernetes makes it real.



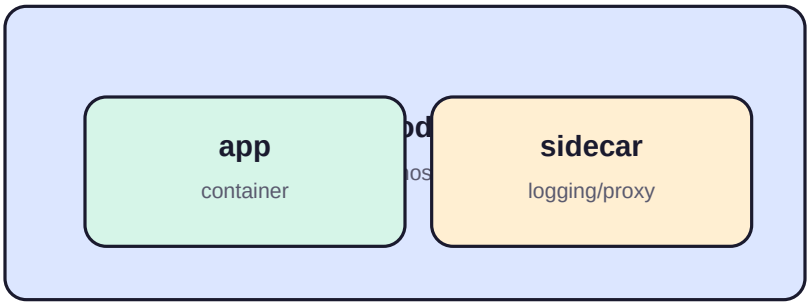
etcd = the brain's memory: every object, every desired state, strongly consistent.

Nodes just run what they're told. If a node dies, the scheduler replaces its pods.

STEP 2/6

Pod: The Smallest Unit

One or more containers sharing network + storage. Ephemeral by design.



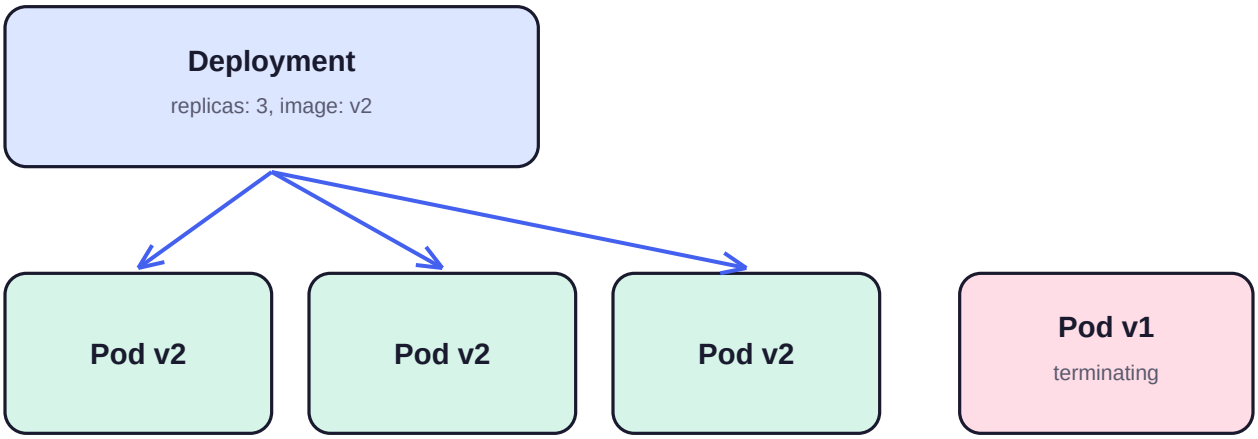
Pods die — never attach to them. Controllers keep the DESIRED COUNT alive.

Sidecars extend pods: service mesh proxies, log shippers, config reloaders.

STEP 3/6

Deployment: Declarative Rollouts

Say '3 replicas of v2' — the controller converges reality.



Rolling update: new pods come up, old pods drain — zero downtime by default.

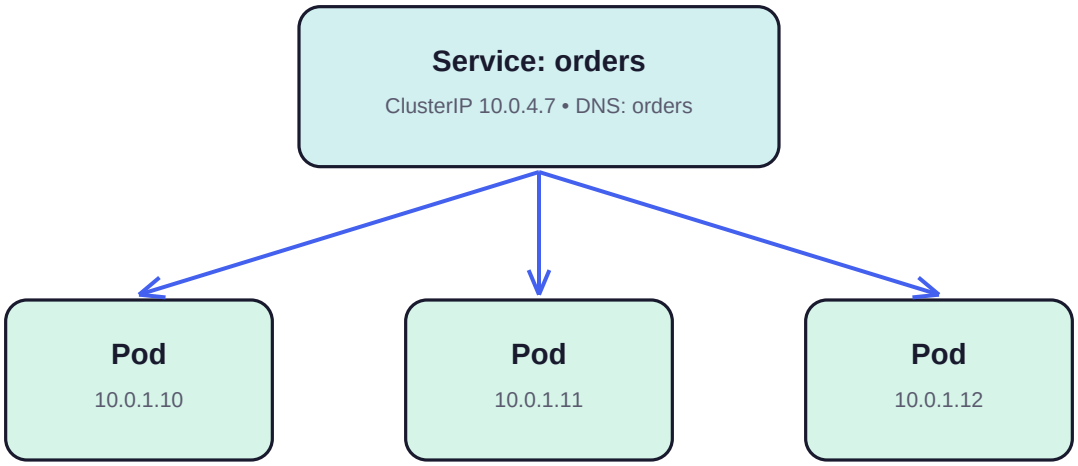
Rollback = set image back to v1. The controller reverses the rollout.

Readiness probes gate traffic; liveness probes restart stuck containers. Get these right.

STEP 4/6

Service: Stable Address for Ephemeral Pods

Pods churn; the Service IP and DNS name don't.



Clients call the Service; kube-proxy load-balances to healthy pods.

Types: ClusterIP (internal), NodePort, LoadBalancer (cloud LB), Ingress (HTTP routing).

STEP 5/6

Config & Secrets: Keep Config Out of Images

Same image, different environments.

ConfigMap

Non-secret config: env vars, config files. Mount as env or volume.

Secret

Sensitive data, base64-encoded (encrypt etcd at rest!).

Volumes

emptyDir (ephemeral), PVC (persistent), config/secret mounts.

Namespaces

Virtual clusters: dev / staging / prod isolation + quotas.

STEP 6/6

Autoscaling in Kubernetes

Two dimensions: more pods, bigger nodes.

HPA

Horizontal Pod Autoscaler■more replicas on CPU/RAM/custom

Cluster Autoscaler

adds/removes NODES■when pods can't schedule

- HPA: 'keep avg CPU at 60% — add pods as needed'
- VPA: right-size requests/limits from history
- Cluster autoscaler: provisions cloud VMs when pending pods pile up
- Scale-to-zero (Knative/KEDA): event-driven, pay-per-use

STEP ✓I✓

Key Takeaways

Kubernetes — Remember

- Control plane declares desired state; nodes run it. etcd is the source of truth.
- Pods are ephemeral — Deployments keep the right count alive with rolling updates.
- Services give stable IPs/DNS to churning pods; Ingress routes HTTP.
- ConfigMaps/Secrets separate config from images; namespaces isolate environments.
- Autoscale pods with HPA, nodes with the cluster autoscaler.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev